Image Painter: An Optimized Stroke-Based Algorithm for Artistic Image Stylization

Chang-Chieh Cheng, *National Yang Ming Chiao Tung University, Hsinchu, 30010, Taiwan*

This article introduces Image Painter (IP), a novel rule-based stroke-based rendering (SBR) algorithm that transforms photographs into painterly images using a sequence of brush strokes. IP employs a principled stroke initialization method combining connected-component labeling and principal component analysis, followed by a dual-stage optimization framework—forward optimization and backward optimization—to refine strokes efficiently and interpretably. Experimental results demonstrate that IP outperforms state-of-the-art SBR methods across key metrics, including mean squared error, structural similarity index measure, peak signal-to-noise ratio, and learned perceptual image patch similarity, even with a limited number of strokes. IP supports diverse painting styles, such as oil sketch, watercolor, pastel, and spray painting, and has practical applications in digital art, step-by-step painting instruction, and synthetic dataset generation for learning-based SBR methods. These results highlight IP's effectiveness, flexibility, and potential for both creative and educational applications.

Stroke-based rendering (SBR) is a nonphotorealistic rendering technique that simulates the process of painting by composing images with sequences of brush strokes.[5] It enables image stylization by substituting strokes with custom brush patterns and achieves image abstraction by using a limited number of strokes.

The core challenge in SBR lies in placing strokes effectively while optimizing rendering parameters, such as color, position, and size. This is typically framed as an optimization problem that minimizes an objective function measuring the difference between the rendered and input images. Several learning-based SBR (LSBR) approaches have been proposed, including learning to paint (LP),[8] paint transformer (PT),[10] stylized neural painting (SNP),[20] and compositional neural painter (CNP).[7] These methods can produce high-quality renderings but often rely on large training datasets, lack interpretability, and may struggle to maintain structural fidelity with very few strokes.

In contrast, rule-based SBR (RSBR) methods rely on hand-crafted rules instead of learned models or training data. Although typically slower than LSBR approaches, RSBR offers notable advantages in robustness, generalization, and user controllability.[17]

RSBR techniques can be grouped into Voronoi-based methods, which partition the image into regions and place strokes at region centers,[6] and trial-and-error methods, which iteratively optimize strokes to minimize an objective function.[2] Notably, some RSBR approaches have found applications beyond traditional image stylization. For example, GEIR (Gaussian ellipse image representation and reconstruction),[2] which represents strokes using 2-D Gaussians, has been applied for denoising in tomographic reconstruction with sparse-view projections, demonstrating the broader utility of stroke-based representations in scientific imaging. Despite such advances, both LSBR and RSBR methods often fail to accurately reproduce images with fewer than 500 strokes, particularly in preserving fine structural

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-No Derivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/
Digital Object Identifier 10.1109/MMUL.2025.3646865
Date of publication 22 December 2025; date of current version 23 March 2026.

details. This reveals a persistent knowledge gap: Existing approaches emphasize global style or texture over structural fidelity, limiting practical applications where computational efficiency or low-stroke abstraction is required.

This article introduces Image Painter (IP), a novel RSBR algorithm that addresses these gaps. IP produces high-quality painterly renderings while preserving essential structures, even under severe stroke count constraints. It is applicable in practical scenarios, such as educational tools for guiding novice painters, multimedia content generation, and augmentation for training LSBR systems. Figure 1 illustrates IP progressively refining an image with stroke counts ranging from 50 to 2000, adapting to varying levels of detail. The input image, sourced from ImageNet,[3] is rendered in an oil-painting style with 50, 100, 250, 500, and 2000 strokes.

The IP pipeline consists of three main stages:

1) *Stroke initialization (SI):* Differences between the input and output images are analyzed to initialize stroke parameters.
2) *Forward optimization (FO):* A local search algorithm iteratively refines each stroke to minimize the objective function.
3) *Backward optimization (BO):* After all strokes are placed, the process is reversed to further refine them and enhance visual fidelity.

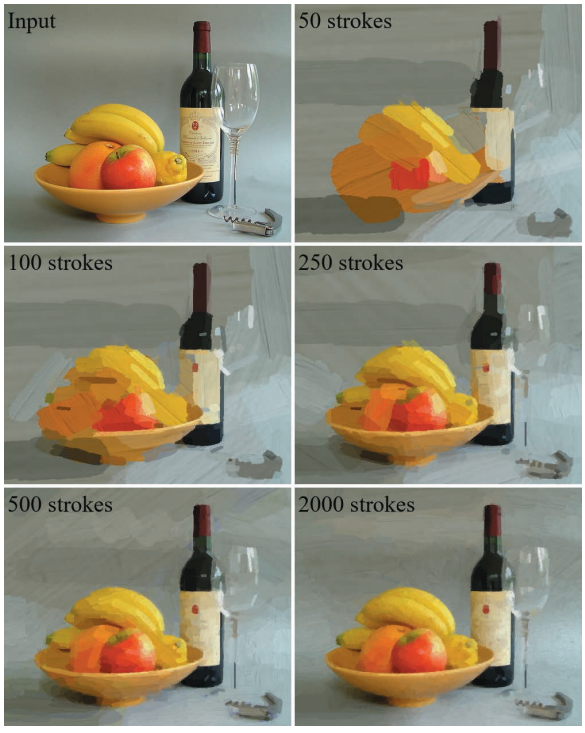


FIGURE 1. Oil painting simulations generated by the proposed method. The first image is the input photograph. Subsequent images illustrate the outcomes achieved by the proposed method utilizing 50, 100, 250, 500, and 2000 strokes, respectively.

Compared with previous studies, IP advances existing SBR methods through four main contributions:

- *Accurate rendering with few strokes:* IP achieves high structural fidelity even under tight stroke constraints, outperforming both LSBR and RSBR methods in low-stroke scenarios.
- *Principled SI:* IP integrates connected-component labeling (CCL)[16] and principal component analysis (PCA)[4] to more effectively capture local structural features than previous approaches.
- *Dual-stage optimization framework:* The FO–BO pipeline introduces an interpretable optimization process that balances sparsity and smoothness using an ℓ_1–ℓ_2 objective function.
- *Practical applicability:* IP supports efficient rendering and is suitable for multimedia content creation and educational applications, with stable computational and memory performance across high-resolution images.

Extensive experiments validate IP's effectiveness, demonstrating superior structural fidelity, rendering quality, and efficiency, particularly under tight stroke budgets. Furthermore, IP addresses open research challenges in RSBR, including interpretable optimization and the tradeoff between stroke count, detail preservation, and computational cost, providing a foundation for future enhancements, such as adaptive stroke strategies and integration with real-time applications.

PRELIMINARIES

In RSBR algorithms, each stroke is typically defined by a set of rendering parameters, including position, size, orientation, and color.[17] Some methods further include opacity to simulate transparency effects.[2] The initialization of these parameters strongly affects both convergence behavior and rendering quality, as poor initialization can lead to suboptimal local minima.[15] Thus, effective SI is a key determinant of visual fidelity and computational performance in RSBR systems.

Traditional SBR methods often rely on multiscale pyramid representations,[13] generating strokes progressively from coarse to fine to emulate human painting. Others, such as Obaid et al.,[14] employ CCL to segment images into regions of uniform color and initialize

strokes using region centroids and mean colors.[4] However, such methods may not adequately capture the dominant orientation or geometric distribution of structures within each region.

Recent RSBR techniques, including Im2Oil[17] and GEIR,[2] improve initialization through feature-driven or probabilistic strategies, producing more structured stroke placement. Nonetheless, these methods still face challenges in maintaining high fidelity when using few strokes or in balancing computational cost and structural precision.

The proposed IP extends these ideas in three important ways:

- It enhances CCL-based initialization by integrating PCA to accurately determine each stroke's position, size, and dominant orientation.
- It adopts a hybrid optimization framework derived from GEIR, redefining the objective function with combined ℓ_1–ℓ_2 norms to balance detail preservation and smoothness.
- It introduces a dual-stage refinement process—FO and BO—that iteratively adjusts stroke parameters through both forward and reverse compositing, achieving fine-grained control and improved convergence stability.

This combination allows IP to maintain structural integrity even under tight stroke constraints, addressing a key gap in existing RSBR approaches that often degrade in low-stroke scenarios. Detailed procedures of FO and BO are presented in the "Forward Optimization" and "Backward Optimization" sections.

METHOD

An overview of the proposed method, IP, is illustrated in Figure 2. Let $T(\mathbf{x}) \in \mathbb{R}^3$ denote the input RGB image with $\eta = \eta_x \times \eta_y$ pixels, where $x \in \mathbb{R}^2$ represents the pixel coordinates, and η_x and η_y are the width and height of T, respectively. The parameters m and n indicate the maximum number of strokes and the current stroke count, respectively, with n initialized to zero.

Step 1 performs SI by analyzing the difference between T and the intermediate painting In to generate a set of b new strokes, denoted as $\mathbf{Q}_b$. Here, I_n is the image rendered from the current n strokes (stroke rendering), and I_0 assigns a uniform initial color to all pixels. Step 2 applies FO to refine the rendering parameters of the newly added strokes $\mathbf{Q}_b$. Step 3 renders $\mathbf{Q}_b$ onto I_n, and the stroke count n is updated accordingly. Steps 1–3 are repeated until n reaches m, producing a total of m optimized strokes, denoted as $\mathbf{Q}_m$. To further enhance rendering quality, Step 4 performs BO ω times to iteratively refine $\mathbf{Q}_m$ in reverse order. Finally, in Step 5, the complete set of strokes $\mathbf{Q}_m$ is rendered onto the final output image I_m.

Stroke Rendering

Let $F(\mathbf{x})$ and $G(\mathbf{x})$ represent the intensity and opacity patterns of a stroke, respectively, where both functions are defined over the domain [0, 1]. Rendering a stroke

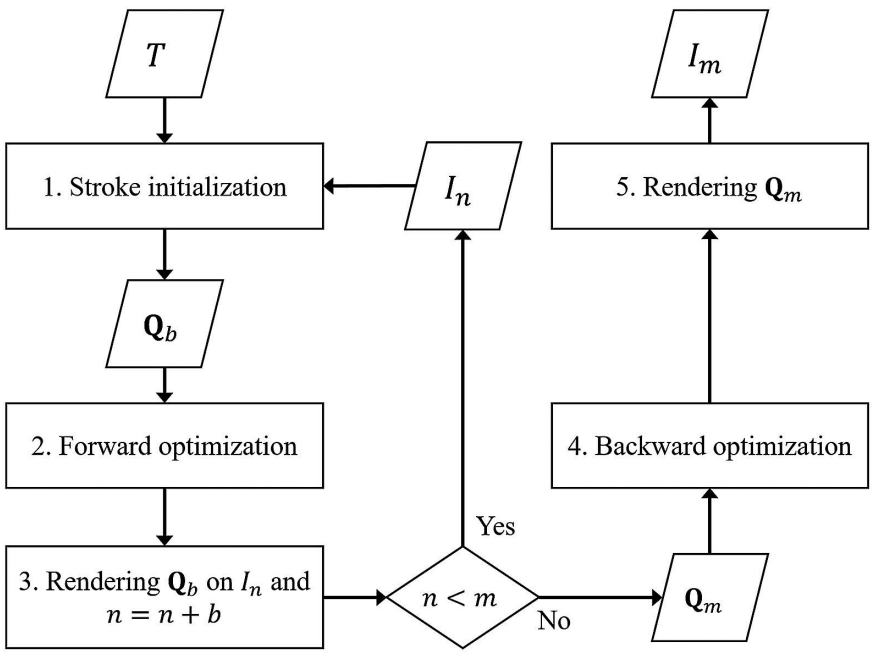


FIGURE 2. Overview of IP. T: input image; $\mathbf{Q}_m$: set of m strokes; I_m: rendering after m strokes.

onto the output image involves employing a rendering parameter vector $\mathbf{q} = [\mathbf{u}, \mathbf{s}, \theta, \alpha, \mathbf{c}]$ to transform and composite F and G. In this vector, $\mathbf{u}$ and $\mathbf{s} \in \mathbb{R}^2$ represent the center location and scale factor, respectively; $0 \leq \theta < \pi$ and $0 \leq \alpha < 1$ indicate the orientation and opacity, respectively; and $\mathbf{c} \in \mathbb{R}^3$ represents the color with red, green, and blue channels. A sequence of n strokes is represented as $\mathbf{Q}_n = \{\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n\}$, where each stroke $\mathbf{q}_k = [\mathbf{u}_k, \mathbf{s}_k, \theta_k, \alpha_k, \mathbf{c}_k]$ for $1 \leq k \leq n$. The stroke rendering process updates the image sequentially according to the following compositing equation:

$$I_k(\mathbf{x}) = \alpha_k F(\mathbf{x}'_k)\mathbf{c}_k + (1 - \alpha_k G(\mathbf{x}'_k))I_{k-1}(\mathbf{x}), \forall \mathbf{x} \quad (1)$$

where $\mathbf{x}'_k = \mathbf{s}_k \odot (R_k(\mathbf{x} - \mathbf{u}_k))$ with the element-wise multiplication $\odot$, and the rotation matrix R_k is defined by

$$R_k = \begin{bmatrix} \cos\theta_k & -\sin\theta_k \\ \sin\theta_k & \cos\theta_k \end{bmatrix}. \quad (2)$$

The initial image I_0 is uniformly initialized such that each pixel is assigned the most frequent color value found in the input image T, following the approach of GEIR.[2]

Stroke Initialization

The initialization of $\mathbf{q}$ consists of three main steps: 1) computing an error map between the current canvas I_n and the target image T; 2) identifying candidate stroke regions based on the error distribution; and 3) extracting rendering parameters from these regions. These steps are illustrated in Figure 3.

The first step computes the disparity map D_n between the current canvas I_n and the target image T, defined as

$$D_n(\mathbf{x}) = \left(G_\sigma * \frac{\| I_n - T \|_1}{3}\right)(\mathbf{x}), \quad \forall \mathbf{x} \quad (3)$$

where I_n and T are normalized to the range [0, 1]. The operator * denotes convolution, and G_σ is a Gaussian kernel with standard deviation σ for noise suppression (set to $\sigma = 1$ in this study).

An error map H_n is then obtained by applying binary thresholding to D_n:

$$H_n(\mathbf{x}) = \begin{cases} 1, & \text{if } D_n(\mathbf{x}) > \beta; \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where β denotes the threshold value, empirically set between 0.25 and 0.35 (see the "Evaluation of SI and Optimization" section).

Figure 3(a) shows a target image T of size $n = 334 \times 500$ pixels from ImageNet. Figure 3(b) displays the intermediate canvas I_n after n strokes rendered using the oil-painting stroke pattern from CNP.[7] The corresponding disparity map D_n and binary error map H_n are shown in Figure 3(c) and (d), respectively.

The second step uses CCL to group all non-zero pixels in H_n into b distinct stroke regions, denoted as $\mathbf{H}_b = \mathbf{h}_1, \mathbf{h}_2, \cdots, \mathbf{h}_b$. Each region $\mathbf{h}_i$ contains more than $\tau_{\min}$ adjacent non-zero pixels, with $\tau_{\min}$ set to 3 in this study.

As illustrated in Figure 3(d), CCL assigns each set of adjacent white pixels to a stroke region, resulting in the creation of $\mathbf{H}_b$. In particular, two example regions—highlighted in blue and red in Figure 3(d)—correspond to $\mathbf{h}_1$ and $\mathbf{h}_2$, respectively.

The third step extracts the stroke parameter vector $\mathbf{q}$ from each region $\mathbf{h}$. Figure 3(e) illustrates this process for $\mathbf{h}_1$, where all pixels in the region are plotted on a grid and represented as circles. The grid origin is aligned with the center of the bounding box enclosing $\mathbf{h}_1$. The stroke position $\mathbf{u}$ is computed as the centroid of the region, given by the mean of all pixel coordinates in $\mathbf{h}$:

$$\mathbf{u} = \frac{1}{|\mathbf{h}|} \sum_{\mathbf{x} \in \mathbf{h}} \mathbf{x}. \quad (5)$$

PCA is then employed to estimate the primary and secondary axes of $\mathbf{h}$,[4] denoted by $\mathbf{v}$ and $\mathbf{w}$, respectively. Let s_x and s_y represent the spatial ranges $\mathbf{h}$ projected onto $\mathbf{v}$ and $\mathbf{w}$, respectively. The stroke scale vector $\mathbf{s}$ is then defined as

$$\mathbf{s} = \begin{bmatrix} s_x \\ s_y \end{bmatrix}. \quad (6)$$

The stroke orientation θ is computed from the primary axis $\mathbf{v}$ as

$$\theta = \arccos(v_x) \quad (7)$$

where v_x denotes the x-component of $\mathbf{v}$. As depicted in Figure 3(e), a red rectangle with dimensions $s_x \times s_y$ and orientation θ tightly encloses the region $\mathbf{h}_1$.

The stroke color, $\mathbf{c}$, is initialized as the average color of the corresponding region in T:

$$\mathbf{c} = \frac{1}{|\mathbf{h}|} \sum_{\mathbf{x} \in \mathbf{h}} T(\mathbf{x}). \quad (8)$$

In addition, the opacity α is initialized to a specified constant, typically set to 0.9.

However, the second step may generate excessively large stroke regions, which can lead to inaccurate estimates of the initial rendering parameters in the third step. To address this issue, the bounding box of each region $\mathbf{h}$ is subdivided into smaller overlapping subregions with dimensions $\tau = \tau_x \times \tau_y$ pixels, whenever the bounding box size exceeds τ. These subregions are uniformly arranged within the bounding box of $\mathbf{h}$, with each neighboring subregion overlapping by $\tau/2$ pixels to ensure continuity. Let $\mathbf{z}_{\min}$ and $\mathbf{z}_{\max}$ denote the minimum and maximum points of a bounding

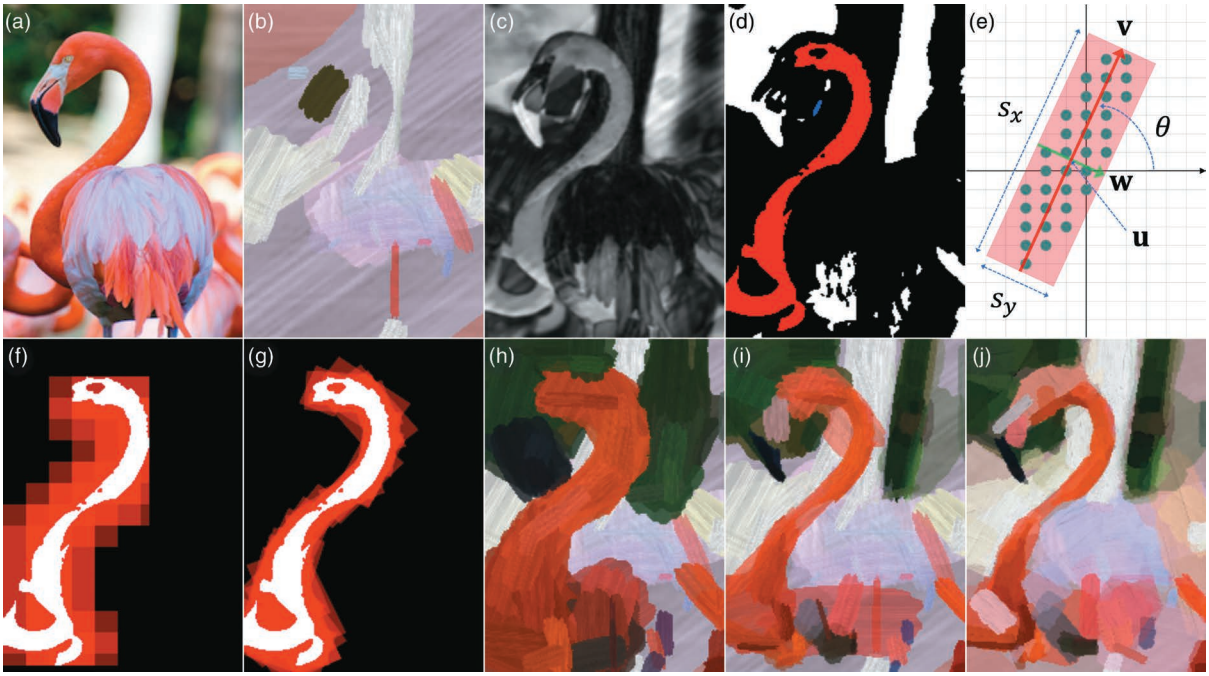


FIGURE 3. Stroke initialization. (a) T, (b) I_n, (c) D_n, and (d) H_n with blue and red regions representing $\mathbf{h}_1$ and $\mathbf{h}_2$, respectively. (e) PCA applied to determine rendering parameters from $\mathbf{h}_1$. (f) Subdivision of excessively large stroke region $\mathbf{h}_2$ into subregions. (g) PCA applied to each subregion of $\mathbf{h}_2$. (h) I_{n+b} before FO. (i) I_{n+b} after FO. (j) I_{n+b} after BO with $\omega = 1$.

box enclosing $\mathbf{h}$. The top-left corner of subregion $\mathbf{z}_{i,j}$ located at the ith row and jth column is defined as

$$\mathbf{z}_{i,j} = \mathbf{z}_{\min} + 0.5\begin{bmatrix}(j-1)\tau_x \\ (i-1)\tau_y\end{bmatrix} \tag{9}$$

where $i \geq 1$, $j \geq 1$, and $\mathbf{z}_{i,j} \leq \mathbf{z}_{\max}$. In this study, the subregion size is set to $\tau = 64 \times 64$ (see the "Evaluation of SI and Optimization" section). As shown in Figure 3(d), $\mathbf{h}_2$ represents a large stroke region. To handle this, the bounding box of $\mathbf{h}_2$ is uniformly divided into multiple subregions, each measuring 64×64 pixels. Adjacent subregions overlap by either 32×64 or 64×32 pixels, depending on their orientation. These subregions—indicated by red boxes in Figure3(f)—are individually processed using the third step to initialize the stroke parameters $\mathbf{q}$, as illustrated in Figure 3(g).

After SI, $\mathbf{Q}_b$ is generated. Subsequently, (1) is utilized to render $\mathbf{Q}_b$ onto I_n, producing I_{n+b}, as illustrated in Figure 3(h). To further enhance rendering quality, the preliminary stroke set $\mathbf{Q}_b$ is refined using the FO and BO procedures, described in detail in the following sections. Figure 3(i) and (j) show the refinement results obtained after applying FO and BO with ω = 1, respectively. As illustrated, FO effectively adjusts the size, color, and orientation of each stroke to better approximate the target structure, while BO further refines stroke placement and opacity, thereby enhancing fine-grained details and overall visual coherence in the final painting.

Forward Optimization

IP utilizes the FO method from GEIR to refine $\mathbf{Q}_b$.[2] Each stroke's rendering parameters $\mathbf{q}_k$ are iteratively updated using either the hill climbing algorithm[9] or the gradient descent method.[1] The update step for $\mathbf{q}_k$ follows

$$\mathbf{q}_k \approx \mathbf{q}_k - \lambda \frac{dE}{d\mathbf{q}_k} \tag{10}$$

where E denotes the objective function, λ represents the step size, and $dE/d\mathbf{q}_k = (dE/dI_k)(dI_k/d\mathbf{q}_k)$. Unlike GEIR, which models strokes using a 2-D Gaussian function, IP adopts a more general formulation involving F and G. As a result, the gradient $dI_k/d\mathbf{q}_k$ must explicitly incorporate both F and G, and is computed as:

$$\frac{dI_k}{d\mathbf{q}_k} = \begin{bmatrix}\frac{\partial I_k}{\partial \mathbf{u}_k} \\ \frac{\partial I_k}{\partial \mathbf{s}_k} \\ \frac{\partial I_k}{\partial \theta_k} \\ \frac{\partial I_k}{\partial \alpha_k} \\ \frac{\partial I_k}{\partial \mathbf{c}_k}\end{bmatrix} = \begin{bmatrix}\alpha_k\left(\frac{\partial F}{\partial \mathbf{u}_k}\mathbf{c}_k - \frac{\partial G}{\partial \mathbf{u}_k}I_{k-1}\right) \\ \alpha_k\left(\frac{\partial F}{\partial \mathbf{s}_k}\mathbf{c}_k - \frac{\partial G}{\partial \mathbf{s}_k}I_{k-1}\right) \\ \alpha_k\left(\frac{\partial F}{\partial \theta_k}\mathbf{c}_k - \frac{\partial G}{\partial \theta_k}I_{k-1}\right) \\ F\cdot\mathbf{c}_k - G\cdot I_{k-1} \\ \alpha_k F\end{bmatrix} \tag{11}$$

where the partial derivatives of F and G are estimated through numerical differentiation.[12]

While GEIR defines its objective function using a combination of the ℓ^1 norm and the structural similarity index measure (SSIM) to enhance detail reconstruction,[18] IP adopts a modified objective to reduce computational overhead. Specifically, IP combines the ℓ^1 and ℓ^2 norms to form the objective function

$$E(I_k, T) = \frac{1}{2\eta}\sum_{\forall \mathbf{x}}(\| I_k(\mathbf{x}) - T(\mathbf{x}) \|_1 + \| I_k(\mathbf{x}) - T(\mathbf{x}) \|_2^2). \quad (12)$$

This formulation provides a practical tradeoff between computational efficiency and the preservation of visual fidelity during the stroke rendering process.

Backward Optimization

After the generation of m strokes, IP applies the BO method from GEIR to further refine $\mathbf{Q}_m$. Following (1), I_m can be expanded recursively as

$$I_m(\mathbf{x}) = I_{k+1,m}(\mathbf{x}) + \alpha_k A_k(\mathbf{x}) F(\mathbf{x}'_k)\mathbf{c}_k + A_{k-1}(\mathbf{x}) I_{k-1}(\mathbf{x}) \quad (13)$$

where $0 < k \leq m$,

$$I_{k+1,m}(\mathbf{x}) = \sum_{i=k+1}^{m} A_i \alpha_i F(\mathbf{x}'_i)\mathbf{c}_i \quad (14)$$

and

$$A_i(\mathbf{x}) = \begin{cases} \prod_{j=i+1}^{m} 1 - \alpha_j G(\mathbf{x}'_j), & \text{if } 1 \leq i < m; \\ 1, & \text{otherwise} \end{cases}. \quad (15)$$

Accordingly, BO refines each $\mathbf{q}_k$ by applying (13) and iteratively updating the parameters ω times using either the hill-climbing algorithm or gradient descent. The update process relies on the following partial derivatives of Im with respect to $\mathbf{q}_k$:

$$\frac{dI_m}{d\mathbf{q}_k} = \begin{bmatrix} \frac{\partial I_m}{\partial \mathbf{u}_k} \\ \frac{\partial I_m}{\partial \mathbf{s}_k} \\ \frac{\partial I_m}{\partial \theta_k} \\ \frac{\partial I_m}{\partial \alpha_k} \\ \frac{\partial I_m}{\partial \mathbf{c}_k} \end{bmatrix} = A_k \begin{bmatrix} \alpha_k\left(\frac{\partial F}{\partial \mathbf{u}_k}\mathbf{c}_k - \frac{\partial G}{\partial \mathbf{u}_k} I_{k-1}\right) \\ \alpha_k\left(\frac{\partial F}{\partial \mathbf{s}_k}\mathbf{c}_k - \frac{\partial G}{\partial \mathbf{s}_k} I_{k-1}\right) \\ \alpha_k\left(\frac{\partial F}{\partial \theta_k}\mathbf{c}_k - \frac{\partial G}{\partial \theta_k} I_{k-1}\right) \\ F \cdot \mathbf{c}_k - G \cdot I_{k-1} \\ \alpha_k F \end{bmatrix}. \quad (16)$$

$$= A_k \frac{dI_k}{d\mathbf{q}_k}$$

EXPERIMENTS AND RESULTS

IP was implemented using Python version 3.11.7, with PyTorch version 2.1.2 utilized for computational tasks. The source code of IP is publicly available at https://github.com/jameschengcs/ip. Experimental validation of IP was conducted through three distinct experiments, as detailed in subsequent sections. The test set included 1500 photographs from ImageNet,[3] 1500 portraits from CelebA,[11] and 300 high-resolution, detail-rich photographs from a self-collected dataset covering portraits, landscapes, buildings, vehicles, and other everyday objects. Each test image was resized to 512×512 pixels for processing, and then restored to its original resolution for output. Evaluation of rendering quality employed four metrics: mean squared error (MSE), SSIM, peak signal-to-noise ratio (PSNR), and learned perceptual image patch similarity (LPIPS) version 0.1.2.[19] Higher SSIM and PSNR values, combined with lower MSE and LPIPS scores, collectively indicate superior rendering quality. All experiments were performed on a computing system featuring an Intel i9 3.7-GHz processor, 64 GB of memory, and an NVIDIA RTX 3080 GPU.

Evaluation of SI and Optimization

To assess the effectiveness of SI, we compared it against a baseline method where each $\mathbf{q}$ of a stroke was randomly initialized (referred to as Random). In the Random method, each parameter was uniformly sampled within its valid range. The configuration of SI in this experiment is listed as follows: $\sigma = 1$, $\beta = 0.3$, $\tau_{\min} = 3$, and $\tau = 64 \times 64$. The stroke pattern F utilized in this experiment was an oil brush pattern obtained from GEIR,[2] and

$$G(\mathbf{x}) = \begin{cases} 0, & \text{if } F(\mathbf{x}) \leq 0, \forall \mathbf{x}; \\ 1, & \text{otherwise} \end{cases}. \quad (17)$$

Table 1 compares the performance of various SI methods for $m = 250$ and 500. Each metric represents the average over the test dataset. The reported time values (in seconds) indicate the duration between the listed step and the preceding one. The rows labeled CCL and PCA under the Step column reflect the individual contributions of the CCL and PCA components within SI. In CCL, for each stroke $\theta = 0$, and the vectors $\mathbf{v}$ and $\mathbf{w}$ are aligned with the x- and y-axes, respectively, for computing (6). PCA provides a slight improvement in stroke alignment and scaling by estimating the dominant orientation and aspect ratio of each region. As shown in Table 1, the initial stroke parameters generated by SI's CCL and PCA outperform those produced by the Random strategy (denoted as INI in the Step column), although they require approximately three times more computation time.

FO was subsequently applied to optimize each initialized $\mathbf{q}$. Rows in Table 1 labeled FO in the Step column indicate that FO significantly enhanced rendering quality, regardless of whether Random or SI was used

TABLE 1. Performance comparison of SI (INIT), FO, and BO with ω = 1, 2, and 5.

m	INIT	Step	Time(s)	MSE↓	SSIM↑	PSNR↑	LPIPS↓
250	Random	INI	0.4	0.1425	0.2186	8.57	0.7591
		FO	40.1	0.0122	0.5181	19.73	0.45
		$\omega = 1$	26.1	0.0093	0.5639	21.03	0.411
		$\omega = 2$	14.5	0.0086	0.5743	21.41	0.4002
		$\omega = 5$	28.5	0.008	0.5869	21.79	0.3875
	SI	CCL	0.8	0.0644	0.373	12.27	0.6242
		PCA	0.1	0.0593	0.3885	12.76	0.6081
		FO	17.7	0.0078	0.567	21.9	0.401
		$\omega = 1$	20.4	0.0062	0.6072	23.03	0.3585
		$\omega = 2$	11.3	0.0059	0.614	23.25	0.3503
		$\omega = 5$	27.4	0.0057	0.6231	23.49	0.3397
500	Random	INI	0.5	0.1403	0.1915	8.6	0.76
		FO	59.7	0.01	0.5412	20.64	0.4233
		$\omega = 1$	43.4	0.0071	0.6061	22.35	0.3648
		$\omega = 2$	21.4	0.0065	0.6207	22.77	0.3507
		$\omega = 5$	46.3	0.006	0.6366	23.23	0.3318
	SI	CCL	1.1	0.0509	0.3797	13.24	0.6272
		PCA	0.2	0.0501	0.383	13.48	0.614
		FO	30	0.006	0.6098	23.11	0.3517
		$\omega = 1$	33.9	0.0045	0.6576	24.57	0.2982
		$\omega = 2$	18.1	0.0043	0.6657	24.83	0.2887
		$\omega = 5$	46.4	0.004	0.6762	25.12	0.2768

to initialize each **q**. However, FO incurs greater computation time when starting from Random, likely due to the challenges posed by poor initial values, which can hinder optimization performance.[15]

Next, BO was applied to further refine $\mathbf{Q}_m$ using ω rounds. The columns corresponding to $\omega = 1$, $\omega = 2$, and $\omega = 5$ present the rendering results refined by BO with three different rounds. Corresponding rows in Table 1 indicate that BO consistently enhances rendering quality. However, the marginal improvement diminishes as ω increases. Therefore, setting $\omega = 1$ or 2 is recommended to strike a good balance between rendering quality and computational cost.

The parameters τ and β play critical roles in SI, influencing region partitioning and error detection, respectively. Tables 2 and 3 present rendering performance when SI is followed by FO, under varying values of τ and β.

Table 2 presents the results for different values of τ across stroke counts $m = 100$, 250, and 500. Smaller β values result in finer subdivisions of large regions, which can better capture local details but increase computation time. Optimal rendering performance—measured by SSIM and PSNR—is generally achieved when $\tau = 64 \times 64$.

Table 3 explores the effect of β for $m = 100$, 250, and 500. Lower values of β tend to detect more error regions, increasing the number of strokes and computation time. The best rendering quality is typically achieved when β is between 0.25 and 0.35, striking a balance between accuracy and efficiency.

Next, the self-collected dataset containing 300 images was used to validate IP under the settings $\tau = 64 \times 64$, $\beta = 0.3$, and $m \in \{100, 250, 500\}$, with FO applied and BO omitted. Figure 4(a) and (b) show the SSIM and PSNR histograms for the validation dataset, where the

red, green, and blue lines correspond to $m = 100$, 250, and 500, respectively. The bin sizes for SSIM and PSNR were set to 0.2 and 3, respectively. Both SSIM and PSNR exhibit approximately normal distributions. The mean SSIM values for $m = 100$, 250, and 500 were 0.4835, 0.5145, and 0.5368, respectively, while the corresponding mean PSNR values were 19.61, 21.24, and 21.99.

The validation dataset was then randomly divided into five folds, each containing 60 images. Figure 4(c) and (d) show the mean SSIM and PSNR for each fold.

TABLE 2. Performance of SI+FO with varying τ.

m	τ	Time(s)	MSE↓	SSIM↑	PSNR↑	LPIPS↓
100	32 × 32	11.9	0.0203	0.4822	17.48	0.5122
	64 × 64	11.9	**0.0131**	**0.5106**	**19.53**	0.4723
	128 × 128	9.2	0.0132	0.5105	19.51	**0.4653**
	256 × 256	**7.7**	0.0147	0.5102	19.15	0.467
250	32 × 32	18	0.0103	0.5510	20.75	0.4302
	64 × 64	17.7	**0.008**	**0.5688**	**21.8**	**0.3984**
	128 × 128	13.9	0.0087	0.5652	21.53	0.4014
	256 × 256	**13.8**	0.0103	0.5623	21.02	0.4064
500	32 × 32	30	**0.0062**	0.611	22.99	0.3551
	64 × 64	23	**0.0062**	**0.6121**	**23.05**	**0.3505**
	128 × 128	21.7	0.0069	0.608	22.7	0.3556
	256 × 256	**21.6**	0.0085	0.602	22.09	0.3639

Boldface indicates the best result for each metric.

TABLE 3. Performance of SI+FO with varying β.

m	β	Time(s)	MSE↓	SSIM↑	PSNR↑	LPIPS↓
100	0.15	14.9	0.0172	0.4996	18.45	0.4859
	0.2	13.3	0.0154	0.5046	18.89	0.48
	0.25	12.4	**0.0145**	**0.506**	**19.11**	**0.476**
	0.3	11.8	0.0147	0.503	19.06	0.4779
	0.35	11.1	0.0149	0.5036	19.03	0.4776
	0.4	**10.4**	0.0154	0.5034	18.96	0.4778
250	0.15	25.7	0.0123	0.5527	20.25	0.4215
	0.2	22.5	0.0099	0.5614	21.06	0.4098
	0.25	20.9	0.0086	0.5641	21.51	0.4056
	0.3	19.9	**0.0084**	0.5639	**21.61**	0.4056
	0.35	19.0	**0.0084**	**0.5646**	**21.61**	**0.4046**
	0.4	**18.3**	**0.0084**	0.5642	21.6	0.4075
500	0.15	37.4	0.0099	0.5965	21.5	0.372
	0.2	32.7	0.0075	0.6079	22.45	0.3555
	0.25	30.5	0.0063	0.6116	22.98	0.3521
	0.3	29.3	**0.006**	**0.6117**	**23.11**	**0.352**
	0.35	28.4	**0.006**	0.6111	**23.11**	**0.352**
	0.4	**27.7**	**0.006**	0.6109	23.1	0.3541

Boldface indicates the best result for each metric.

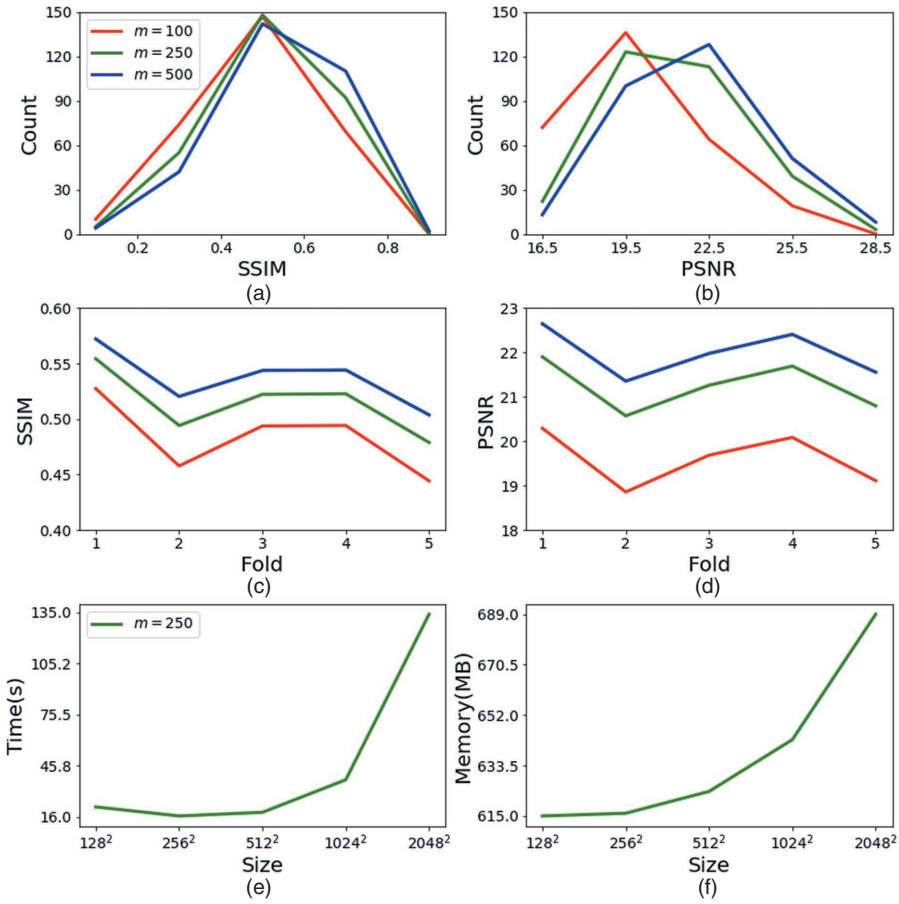


FIGURE 4. Validation of IP on the self-collected dataset. (a) and (b) SSIM and PSNR histograms for $m = 100, 250, 500$. (c) and (d) Mean SSIM and PSNR across five folds. (e) and (f) Runtime and memory for images sized 128^2–2048^2 with $m = 250$.

The SSIM variances for $m = 100$, 250, and 500 were 0.0208, 0.0189, and 0.0173, respectively, while the corresponding PSNR variances were 6.46, 6.17, and 6.15.

The validation confirms that the chosen settings of τ and β yield robust performance. SSIM and PSNR exhibit bounded variability across folds, indicating that these parameters effectively balance stroke size and thresholding to ensure reliable image quality.

Finally, the validation dataset was used to analyze the time and memory consumption of IP under $m = 250$. Each image was resized to five different resolutions: 128^2, 256^2, 512^2, 1024^2, and 2048^2 pixels. Figure 4(e) and (f) show the experimental results. Memory usage grows moderately from 615 MB to 689 MB as image size increases, while processing time rises from 16.5–21.7 s for smaller images to 134 s for the largest resolution. These results indicate that IP maintains stable memory requirements even for high-resolution inputs.

These experiments demonstrate the effectiveness of SI, FO, and BO for painting simulation with a limited number of strokes. The rendering results also demonstrate that IP effectively transforms an input photograph into an oil sketch with $m \leq 500$, while accurately depicting the majority of significant features.

Comparison

The performance of IP was evaluated across different values of m and compared with six recently proposed SBR algorithms, namely LP (2019),[8] PT (2021),[10] SNP (2021),[20] Im2Oil (2022),[17] CNP (2023),[7] and GEIR (2024).[2] It is noteworthy that Im2Oil and GEIR are RSBR algorithms, whereas LP, PT, SNP, and CNP are LSBR methods. In this comparison, F and G were obtained from a palette knife pattern sourced from CNP and (17), respectively, with parameters set as $\sigma = 1, \beta = 0.3$, $\tau_{min} = 3, \tau = 64 \times 64$, and $\omega = 2$. The quantitative comparison results, summarized in Table 4, represent the average performance across all test images. Figures 5, 6, and 7 present representative rendering results from CelebA, ImageNet, and the self-collected dataset, respectively. As shown, IP consistently achieves the highest overall rendering quality across most m, outperforming competing SBR methods

TABLE 4. Performance comparison of variant SBR algorithms.

m	Method	Time(s)	Memory(MB)	MSE↓	SSIM↑	PSNR↑	LPIPS↓
50	LP[8]	7.4	601	0.016	**0.5056**	18.56	0.5371
	PT[10]	**2.2**	591	0.0634	0.3701	12.23	0.5559
	SNP[20]	64.4	814	0.0189	0.4298	17.7	0.5093
	Im2Oil[17]	157.5	14,102	0.0529	0.3616	13.26	0.567
	CNP[7]	3.8	597	0.0192	0.4499	17.64	0.4971
	GEIR[2]	51.7	**334**	0.0288	0.454	15.78	0.4825
	IP	13.4	624	**0.0124**	0.4973	**19.85**	**0.455**
100	LP	7.9	601	0.0124	0.5295	19.77	0.524
	PT	**2.4**	594	0.0528	0.381	13.14	0.5411
	SNP	78	821	0.0124	0.4819	19.62	0.4623
	Im2Oil	113	22,857	0.0435	0.3725	14.16	0.5379
	CNP	4	601	0.0143	0.4664	18.99	0.4713
	GEIR	88.1	**333**	0.0232	0.4739	16.69	0.4454
	IP	19.1	624	**0.009**	**0.5303**	**21.4**	**0.4138**
250	LP	9.8	601	0.01	0.5568	20.87	0.5161
	PT	**2.9**	594	0.038	0.4092	14.48	0.5142
	SNP	122.3	852	0.0085	0.5328	21.37	0.4068
	Im2Oil	85	7,264	0.0311	0.3981	15.74	0.4768
	CNP	4.6	682	0.0118	0.4967	20.01	0.4483
	GEIR	171.9	**335**	0.0173	0.5195	18.05	0.3882
	IP	32.8	621	**0.006**	**0.5966**	**23.06**	**0.3537**
500	LP	13.1	601	0.0094	0.5661	21.11	0.5209
	PT	**3.4**	594	0.0261	0.4419	16.23	0.4826
	SNP	181.4	882	0.0073	0.5601	22.09	0.3802
	Im2Oil	84.1	5,718	0.0264	0.427	16.83	0.4357
	CNP	5.6	682	0.0087	0.5219	21.2	0.4149
	GEIR	263	**333**	0.0149	0.5522	18.71	0.3569
	IP	50.6	624	**0.0047**	**0.6308**	**24.54**	**0.3069**
2000	LP	34	601	0.0095	0.5705	21.15	0.5423
	PT	**4**	595	0.0167	0.5029	18.05	0.449
	SNP	635.2	31,968	0.0062	0.6022	22.69	0.3346
	Im2Oil	115.4	5,325	0.0111	0.5483	20.66	0.3268
	CNP	11.3	683	0.0069	0.5897	22.72	0.3628
	GEIR	505.6	**333**	0.0133	0.585	19.15	0.3341
	IP	108.9	621	**0.0034**	**0.7119**	**26.37**	**0.2309**

Boldface indicates the best result for each metric.

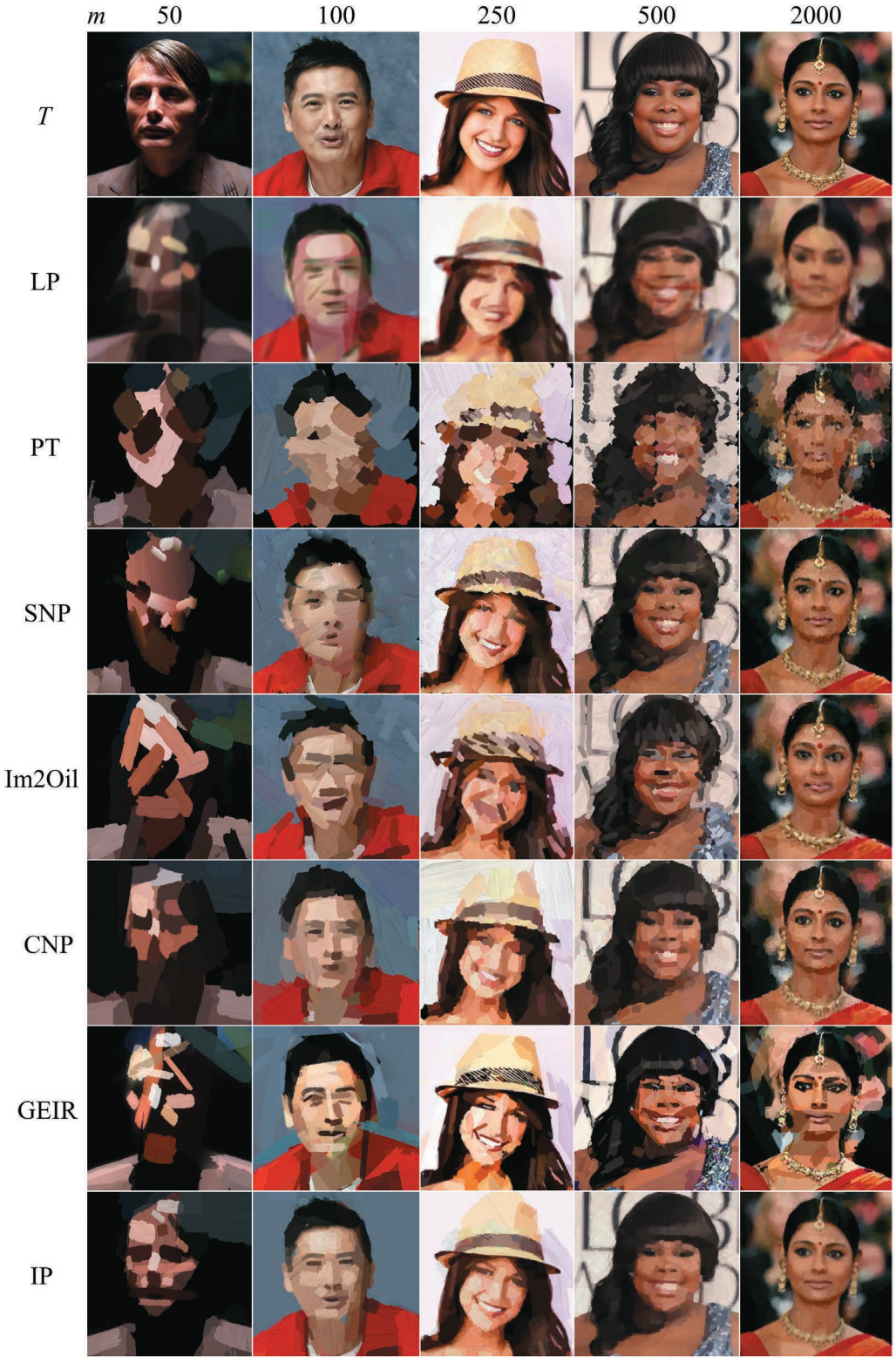


FIGURE 5. Comparison of rendered results on CelebA using various SBR algorithms. Each row presents an input face image followed by renderings from LP, PT, SNP, Im2Oil, CNP, GEIR, and IP. Stroke counts increase from left to right with $m = 50$, 100, 250, 500, and 2000.

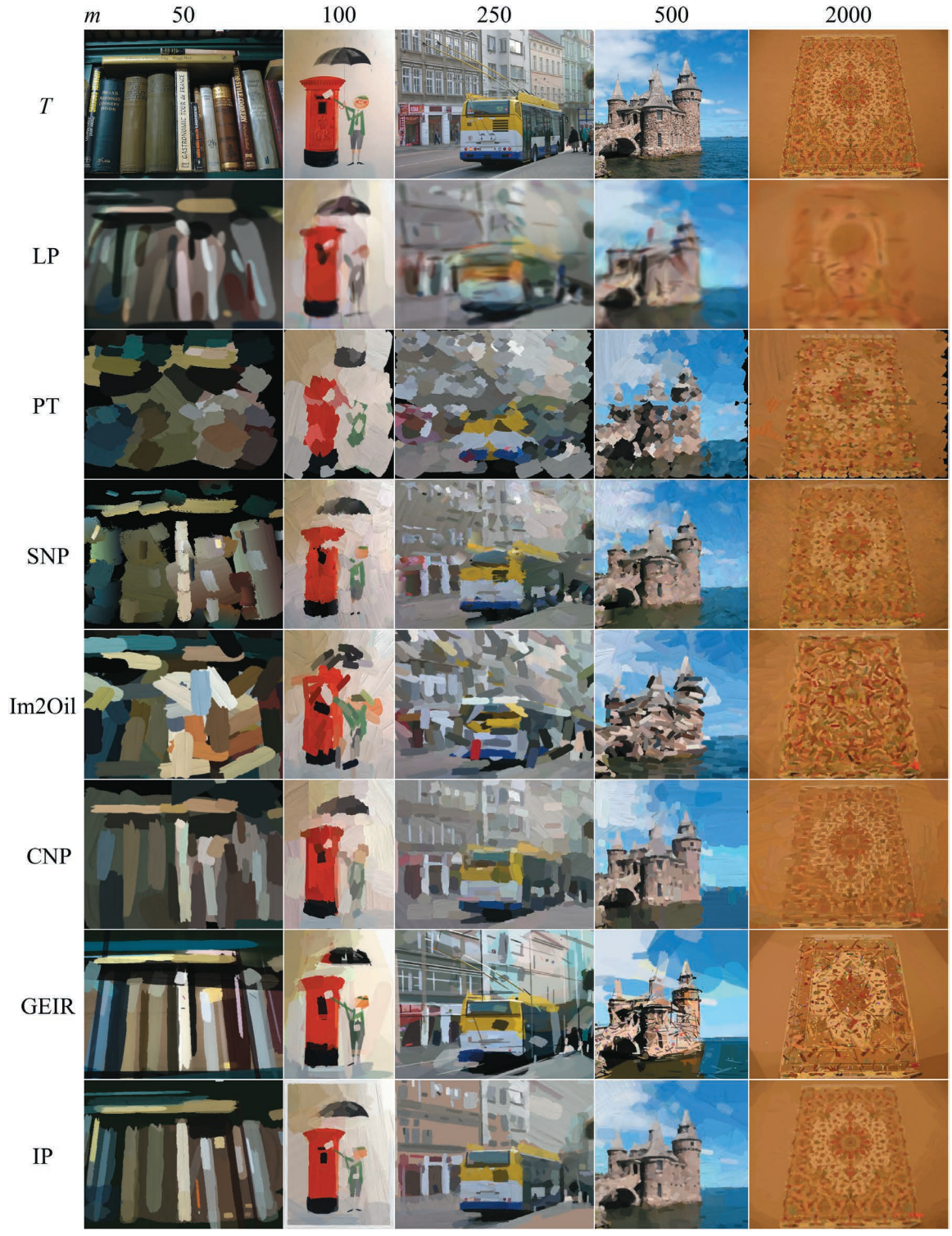


FIGURE 6. Comparison of rendered results on ImageNet using different SBR algorithms. From top to bottom: input images and images rendered by LP, PT, SNP, Im2Oil, CNP, GEIR, and IP. From left to right: varying stroke counts with $m = 50$, 100, 250, 500, and 2000.

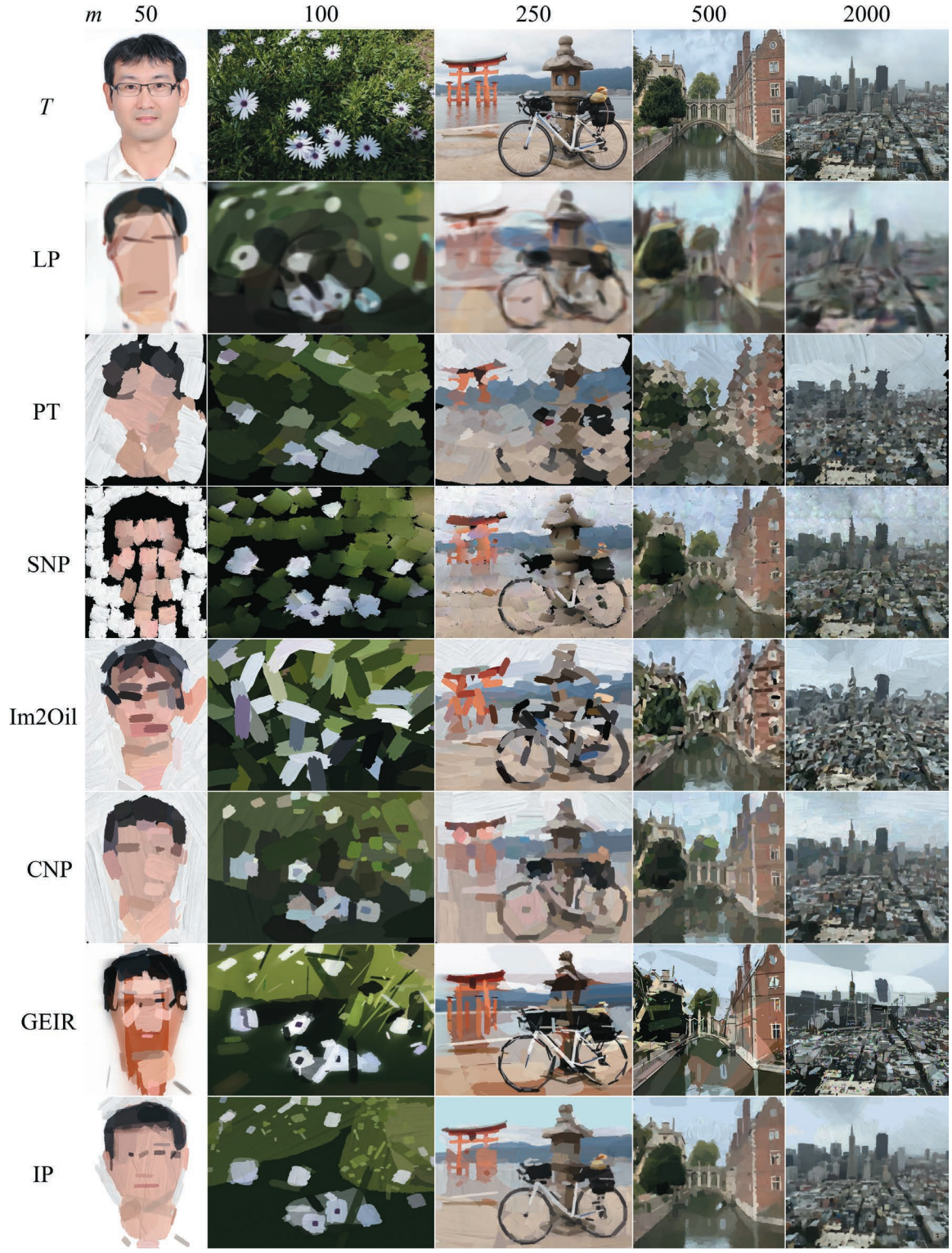


FIGURE 7. Comparison of rendered results on the self-collected dataset using different SBR algorithms. From top to bottom: input images and images rendered by LP, PT, SNP, Im2Oil, CNP, GEIR, and IP. From left to right: varying stroke counts with $m = 50$, 100, 250, 500, and 2000.

in terms of MSE, PSNR, and LPIPS, while attaining competitive SSIM scores. Remarkably, IP produces visually coherent results even with fewer than 100 strokes, effectively preserving essential contours and textures to generate abstract yet recognizable artistic renderings. In contrast, competing methods often fail to maintain structural integrity, resulting in blurred or unrecognizable subjects. Among RSBR algorithms, Im2Oil delivers a pleasing abstract painting effect, whereas GEIR excels in preserving fine structural consistency. Although PT attains the fastest runtime due to its lightweight design, IP demonstrates superior rendering efficiency among RSBR algorithms. In terms of memory consumption, IP maintains a stable footprint of approximately 620 MB, independent of m. By contrast, SNP exhibits a substantial increase in memory usage as m grows, reaching several gigabytes for large stroke counts.

Figure 6 presents rendering results across various content types: books, an abstract painting, a street scene, architecture, and a low-contrast image. While IP achieves visually coherent and detailed reconstructions, it requires a relatively large number of strokes (e.g., $m \geq 2000$) to effectively capture fine structures and textures, highlighting a limitation in its efficiency.



FIGURE 8. Image rendered by IP with 300 strokes using different brush patterns. (a) Input image selected from ImageNet. (b)–(f) Result images rendered by IP using oil brush, watercolor, spots, pastel, and spray stroke patterns, respectively. Each rendered image displays its corresponding stroke pattern displayed in the bottom-left corner.

However, in the low-contrast example (rightmost column of Figure 6), IP preserves the main structure and visual coherence better than other SBR methods, which tend to produce overly blurred or distorted results. This suggests that, despite its stroke-intensive nature, IP maintains robustness in challenging scenarios where contrast is low and structural fidelity is critical.

Variant Stroke Patterns

This experiment used IP to simulate different painting styles with varied stroke patterns, as illustrated in Figure 8. Figure 8(a) presents the input image sourced from ImageNet. The rendered results with five distinct F variants—oil brush, watercolor, spots, pastel, and spray—are depicted in Figure 8(b), (c), (d), (e), and (f), respectively, with $m = 300$. Each rendered image exhibits its corresponding stroke pattern displayed in the bottom-left corner. In Figure 8(c), (17) was utilized to define G. Conversely, in Figures 8(b), (d), (e), and (f), $G(\mathbf{x}) = F(\mathbf{x}), \forall \mathbf{x}$ was used to generate a stroke pattern with nonuniform opacity.

The experimental results highlight the capability of IP to effectively simulate various common painting styles when F and G are appropriately designed.

CONCLUSION

This study presents IP, a novel rule-based approach for transforming photographs into painterly images using multiple brush strokes. IP introduces a principled SI method that accurately configures stroke parameters, followed by a dual-stage refinement process—FO and BO—enabling effective and interpretable refinement. As a result, IP achieves high-quality renderings with significantly fewer strokes while supporting diverse styles, including oil sketch, watercolor, pastel, and spray painting.

Experimental evaluations demonstrate that IP outperforms state-of-the-art LSBR and RSBR methods across multiple metrics—including MSE, SSIM, PSNR, and LPIPS—even under low-stroke conditions (fewer than 250 strokes). Notably, the number of strokes directly corresponds to the level of detail in the resulting painting, allowing users to control abstraction versus fidelity.

IP has practical applications in multimedia and education. For instance, it can guide step-by-step painting instruction by showing intermediate results with a small number of strokes, and it can generate large synthetic datasets to augment learning-based SBR methods.

Despite these advantages, IP has two primary limitations. First, its iterative, rule-based optimization introduces higher computational cost compared to learning-based approaches with rapid inference. Second, although IP considers the level of detail to some

extent, highly textured or intricate regions still require a greater number of strokes to maintain visual fidelity, reducing efficiency in low-stroke scenarios.

These limitations suggest several promising directions for future research: improving the analysis of image detail to adaptively determine the required number of strokes based on user-specified or content-driven detail levels, integrating learning-based acceleration to reduce runtime, and developing dynamic stroke allocation mechanisms for texture-rich areas.

Overall, IP advances SBR by combining structural accuracy, interpretability, and flexibility, providing a solid foundation for both practical use and continued research.

ACKNOWLEDGMENTS

This work was supported by the National Science and Technology Council, Taiwan, under Grant NSTC 113-2221-E-A49-158-MY2.

REFERENCES

1. L. G. Brown, "A survey of image registration techniques," *ACM Comput. Surv.*, vol. 24, no. 4, pp. 325–376, 1992, doi: 10.1145/146370.146374.
2. C.-C. Cheng, "Image representation and reconstruction by compositing Gaussian ellipses," *IET Image Process.*, vol. 18, no. 2, pp. 493–506, 2024, doi: 10.1049/ipr2.12964.
3. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 248–255, 2009, doi: 10.1109/CVPR.2009.5206848.
4. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2006.
5. A. Hertzmann, "A survey of stroke-based rendering," *IEEE Comput. Graph. Appl.*, vol. 23, no. 4, pp. 70–81, Jul./Aug. 2003, doi: 10.1109/MCG.2003.1210867.
6. K. E. Hoff, J. Keyser, M. Lin, D. Manocha, and T. Culver, "Fast computation of generalized Voronoi diagrams using graphics hardware," in *Proc. 26th Annu. Conf. Comput. Graph. Interactive Techn. (SIGGRAPH)*, 1999, pp. 277–286, doi: 10.1145/311535.311567.
7. T. Hu et al., "Stroke-based neural painting and stylization with dynamically predicted painting region," in *Proc. 31st ACM Int. Conf. Multimedia (MM)*, New York, NY, USA: ACM, 2023, pp. 7470–7480.
8. Z. Huang, S. Zhou, and W. Heng, "Learning to paint with model-based deep reinforcement learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 8708–8717, doi: 10.1109/ICCV.2019.00880.
9. S. H. Jacobson and E. Yücesan, "Analyzing the performance of generalized hill climbing algorithms," *J. Heuristics*, vol. 10, no. 4, pp. 387–405, 2004, doi: 10.1023/B:HEUR.0000034712.48917.a9.
10. S. Liu et al., "Paint transformer: Feed forward neural painting with stroke prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 6578–6587, doi: 10.1109/ICCV48922.2021.00653.
11. Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 3730–3738, doi: 10.1109/ICCV.2015.425.
12. J. Modersitzki, *Numerical Methods for Image Registration*. Oxford, U.K.: Oxford University Press, 2003.
13. D. Nehab, and L. Velho, "Multiscale moment-based painterly rendering," in *Proc. XV Brazilian Symp. Comput. Graph. Image Process.*, 2002, pp. 244–251.
14. M. Obaid, R. Mukundan, and T. Bell, "Enhancement of moment based painterly rendering using connected components," in *Proc. Int. Conf. Comput. Graph., Imag. Visualisation (CGIV)*, 2006, pp. 378–383, doi: 10.1109/CGIV.2006.38.
15. R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. 30th Int. Conf. Mach. Learn.*, 2013, vol. 28, pp. 1310–1318.
16. A. Rosenfeld and J. L. Pfaltz, "Sequential operations in digital picture processing," *J. ACM*, vol. 13, no. 4, pp. 471–494, Oct. 1966, doi: 10.1145/321356.321357.
17. Z. Tong, X. Wang, S. Yuan, X. Chen, J. Wang, and X. Fang, "Im2Oil: Stroke-based oil painting rendering with linearly controllable fineness via adaptive sampling," in *Proc. 30th ACM Int. Conf. Multimedia (MM)*, New York, NY, USA: ACM, 2022, pp. 1035–1046.
18. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/tip.2003.819861.
19. R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 586–595, doi: 10.1109/CVPR.2018.00068.
20. Z. Zou, T. Shi, S. Qiu, Y. Yuan, and Z. Shi, "Stylized neural painting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 15,684–15,693, doi: 10.1109/CVPR46437.2021.01543.

CHANG-CHIEH CHENG is an assistant professor with the Information Technology Service Center, National Yang Ming Chiao Tung University, Hsinchu, 30010, Taiwan. His research interests include signal processing, computational imaging, and machine learning. Cheng received his Ph.D. degree in computer science and information engineering from National Chiao Tung University. Contact him at jameschengcs@nycu.edu.tw.